

Chat: API REST Miniblog (Node.js + Express + PostgreSQL)

Conversación con Claude — Kevin, estudiante de Análisis y Desarrollo de Software (SENA)

1. Introducción al proyecto

Pregunta: Nuevo proyecto de API REST en Node.js + Express, temática de miniblog.

Respuesta: Se acordó ir resolviendo el proyecto pregunta por pregunta, con código y explicación línea por línea de cada bloque.

2. Uso correcto y eficiente de Pool de pg

Pregunta: ¿Cómo y cuándo usar Pool de pg de forma correcta y eficiente?

¿Qué es Pool?

Pool mantiene un grupo de conexiones abiertas y reutilizables a PostgreSQL, evitando el costo de abrir/cerrar una conexión en cada request.

Instalación

```
npm install pg
```

Configuración (patrón singleton)

```
// db.js
const { Pool } = require('pg');

const pool = new Pool({
  user: process.env.DB_USER,
  host: process.env.DB_HOST,
  database: process.env.DB_NAME,
  password: process.env.DB_PASSWORD,
  port: process.env.DB_PORT,
  max: 10, // numero maximo de conexiones en el pool
  idleTimeoutMillis: 30000, // cierra conexiones inactivas tras 30s
  connectionTimeoutMillis: 5000 // si no logra conectar en 5s, lanza error
});

module.exports = pool;
```

Explicación clave:

- **max: 10:** máximo de conexiones simultáneas; el resto de peticiones esperan en cola.
- **idleTimeoutMillis:** cierra conexiones inactivas tras el tiempo indicado.
- **connectionTimeoutMillis:** si no logra conectar en ese tiempo, lanza error en vez de esperar indefinidamente.

- Un solo **Pool** por aplicación, exportado como singleton.

Consulta simple

```
// controllers/postController.js
const pool = require('../db');

async function getAllPosts(req, res) {
  try {
    const result = await pool.query('SELECT * FROM posts ORDER BY created_at DESC');
    res.status(200).json(result.rows);
  } catch (error) {
    console.error('Error al obtener posts:', error);
    res.status(500).json({ error: 'Error interno del servidor' });
  }
}

module.exports = { getAllPosts };
```

pool.query() toma y devuelve la conexión automáticamente. **result.rows** contiene el array de filas.

Consultas con parámetros (evitar SQL Injection)

```
async function getPostById(req, res) {
  const { id } = req.params;
  try {
    const result = await pool.query(
      'SELECT * FROM posts WHERE id = $1',
      [id]
    );

    if (result.rows.length === 0) {
      return res.status(404).json({ error: 'Post no encontrado' });
    }

    res.status(200).json(result.rows[0]);
  } catch (error) {
    console.error(error);
    res.status(500).json({ error: 'Error interno del servidor' });
  }
}
```

\$1 es un placeholder seguro que **pg** reemplaza por el valor del array, evitando SQL Injection.

Transacciones con **pool.connect()**

```
async function createPostWithLog(req, res) {
  const { title, content, authorId } = req.body;
  const client = await pool.connect(); // tomamos UNA conexion especifica

  try {
    await client.query('BEGIN');

    const postResult = await client.query(
      'INSERT INTO posts (title, content, author_id) VALUES ($1, $2, $3) RETURNING id',
      [title, content, authorId]
    );
    const newPostId = postResult.rows[0].id;

    await client.query(
      'INSERT INTO logs (action, post_id) VALUES ($1, $2)',
      ['post_created', newPostId]
    );

    await client.query('COMMIT');
    res.status(201).json({ id: newPostId });
  } catch (error) {
    // Error handling
  }
}
```

```

    await client.query('ROLLBACK');
    console.error(error);
    res.status(500).json({ error: 'No se pudo crear el post' });

  } finally {
    client.release(); // SIEMPRE devolvemos la conexión al pool
  }
}

```

`pool.connect()` reserva una conexión específica para ejecutar `BEGIN/COMMIT/ROLLBACK` en la misma conexión física. `client.release()` en `finally` evita connection leaks.

Resumen

Situación	Método
Query simple (SELECT, INSERT, UPDATE, DELETE aislado)	<code>pool.query()</code>
Varias queries atómicas (transacción)	<code>pool.connect()</code> + <code>BEGIN/COMMIT/ROLLBACK</code> + <code>client.release()</code>
Crear el Pool	Una sola vez, en archivo separado, como singleton

3. Formato de la `DATABASE_URL`

Pregunta: Explicación del formato:

```
DATABASE_URL=postgresql://user:password@localhost:5432/miniblog
```

Desglose

- `postgresql://` — esquema/protocolo.
- `user` — nombre de usuario de PostgreSQL.
- `:password` — contraseña del usuario.
- `@localhost` — host donde vive el servidor de PostgreSQL.
- `:5432` — puerto por defecto de PostgreSQL.
- `/miniblog` — nombre de la base de datos específica.

Uso con `pg`

```

// db.js
const { Pool } = require('pg');

const pool = new Pool({
  connectionString: process.env.DATABASE_URL,
});

module.exports = pool;

```

Notas importantes

- Nunca subir el `.env` a git (debe estar en `.gitignore`).
 - Caracteres especiales en la contraseña deben codificarse con URL encoding.
 - En producción, muchas plataformas requieren `?sslmode=require` al final de la URL.
-

4. Crear tablas `authors` y `posts` en SQL

Pregunta: ¿Cómo crear tablas de authors y posts en un script de SQL?

```
-- schema.sql

CREATE TABLE authors (
  id SERIAL PRIMARY KEY,
  name VARCHAR(100) NOT NULL,
  email VARCHAR(150) NOT NULL UNIQUE,
  bio TEXT,
  created_at TIMESTAMP NOT NULL DEFAULT NOW()
);

CREATE TABLE posts (
  id SERIAL PRIMARY KEY,
  title VARCHAR(200) NOT NULL,
  content TEXT NOT NULL,
  author_id INTEGER NOT NULL REFERENCES authors(id) ON DELETE CASCADE,
  published BOOLEAN NOT NULL DEFAULT FALSE,
  created_at TIMESTAMP NOT NULL DEFAULT NOW(),
  updated_at TIMESTAMP NOT NULL DEFAULT NOW()
);

-- Índice para acelerar búsquedas de posts por autor
CREATE INDEX idx_posts_author_id ON posts(author_id);
```

Puntos clave:

- **SERIAL PRIMARY KEY**: identificador autoincremental único.
- **UNIQUE** en email: evita duplicados.
- **REFERENCES authors(id) ON DELETE CASCADE**: foreign key con integridad referencial; al borrar un autor se borran sus posts.
- **DEFAULT NOW()**: solo aplica al insertar, no a updates posteriores.
- Índice en `author_id` acelera JOINS y filtros.

Ejecución del script

```
psql -U user -d miniblog -f schema.sql
```

O desde Node.js:

```
// setup-db.js
const fs = require('fs');
const pool = require('./db');

async function runSchema() {
  const sql = fs.readFileSync('./schema.sql', 'utf8');
  try {
    await pool.query(sql);
    console.log('Tablas creadas correctamente');
  } catch (error) {
    console.error('Error al crear las tablas:', error);
  } finally {
    await pool.end();
  }
}
```

```
}  
  
runSchema();
```

5. Estructura correcta de queries CRUD

Pregunta: ¿Cómo saber si las queries CRUD están bien hechas? ¿Qué estructura deben tener?

Checklist

- Parametrizada (siempre `$1`, `$2...`).
- Específica en columnas (evitar `SELECT *` en producción).
- Maneja el caso "no encontrado" (404, no 500).
- Usa `RETURNING` para no hacer una segunda query.
- Valida tipos antes de llegar a la base de datos.
- Usa el status HTTP correcto.
- Está separada en una capa de "modelo".

Estructura de carpetas

```
src/  
  db.js  
  models/  
    postModel.js  
  controllers/  
    postController.js  
  routes/  
    postRoutes.js
```

Modelo (`models/postModel.js`)

```
const pool = require('../db');  
  
// CREATE  
async function createPost({ title, content, authorId, published = false }) {  
  const result = await pool.query(  
    `INSERT INTO posts (title, content, author_id, published)  
    VALUES ($1, $2, $3, $4)  
    RETURNING id, title, content, author_id, published, created_at`,  
    [title, content, authorId, published]  
  );  
  return result.rows[0];  
}  
  
// READ (todos)  
async function getAllPosts() {  
  const result = await pool.query(  
    `SELECT id, title, content, author_id, published, created_at  
    FROM posts  
    ORDER BY created_at DESC`  
  );  
  return result.rows;  
}  
  
// READ (uno solo)  
async function getPostById(id) {  
  const result = await pool.query(  
    `SELECT id, title, content, author_id, published, created_at  
    FROM posts  
    WHERE id = $1`  
  );  
  return result.rows[0];  
}
```

```

        `SELECT id, title, content, author_id, published, created_at
        FROM posts
        WHERE id = $1`,
        [id]
    );
    return result.rows[0] || null;
}

// UPDATE
async function updatePost(id, { title, content, published }) {
    const result = await pool.query(
        `UPDATE posts
        SET title = $1, content = $2, published = $3, updated_at = NOW()
        WHERE id = $4
        RETURNING id, title, content, author_id, published, updated_at`,
        [title, content, published, id]
    );
    return result.rows[0] || null;
}

// DELETE
async function deletePost(id) {
    const result = await pool.query(
        `DELETE FROM posts WHERE id = $1 RETURNING id`,
        [id]
    );
    return result.rows[0] || null;
}

module.exports = { createPost, getAllPosts, getPostById, updatePost, deletePost };

```

Controlador (**controllers/postController.js**)

```

const postModel = require('../models/postModel');

async function create(req, res) {
    const { title, content, authorId, published } = req.body;

    if (!title || !content || !authorId) {
        return res.status(400).json({ error: 'title, content y authorId son obligatorios' });
    }

    try {
        const post = await postModel.createPost({ title, content, authorId, published });
        res.status(201).json(post);
    } catch (error) {
        console.error(error);
        res.status(500).json({ error: 'Error al crear el post' });
    }
}

async function getAll(req, res) {
    try {
        const posts = await postModel.getAllPosts();
        res.status(200).json(posts);
    } catch (error) {
        console.error(error);
        res.status(500).json({ error: 'Error al obtener los posts' });
    }
}

async function getOne(req, res) {
    const { id } = req.params;

    if (isNaN(id)) {
        return res.status(400).json({ error: 'El id debe ser numerico' });
    }

    try {

```

```

    const post = await postModel.getPostById(id);
    if (!post) {
      return res.status(404).json({ error: 'Post no encontrado' });
    }
    res.status(200).json(post);
  } catch (error) {
    console.error(error);
    res.status(500).json({ error: 'Error al obtener el post' });
  }
}

async function update(req, res) {
  const { id } = req.params;
  const { title, content, published } = req.body;

  try {
    const post = await postModel.updatePost(id, { title, content, published });
    if (!post) {
      return res.status(404).json({ error: 'Post no encontrado' });
    }
    res.status(200).json(post);
  } catch (error) {
    console.error(error);
    res.status(500).json({ error: 'Error al actualizar el post' });
  }
}

async function remove(req, res) {
  const { id } = req.params;

  try {
    const deleted = await postModel.deletePost(id);
    if (!deleted) {
      return res.status(404).json({ error: 'Post no encontrado' });
    }
    res.status(204).send();
  } catch (error) {
    console.error(error);
    res.status(500).json({ error: 'Error al eliminar el post' });
  }
}

module.exports = { create, getAll, getOne, update, remove };

```

Status codes

Situación	Status
Post creado exitosamente	201 Created
GET o UPDATE exitoso	200 OK
DELETE exitoso	204 No Content
Recurso no existe	404 Not Found
Datos de entrada inválidos	400 Bad Request
Error inesperado del servidor/BD	500 Internal Server Error

6. Recrear tablas borradas accidentalmente

Pregunta: Se eliminaron las tablas en pgAdmin durante una prueba. ¿Cómo volver a llenarlas?

Opciones para re-ejecutar el schema

- pgAdmin: Query Tool, abrir `schema.sql`, ejecutar con F5.
- Terminal: `psql -U tu_usuario -d miniblog -f schema.sql`
- Node.js: `node setup-db.js`

Script seguro para re-ejecutar (con DROP)

```
-- schema.sql (version segura para re-ejecutar)

DROP TABLE IF EXISTS posts;
DROP TABLE IF EXISTS authors;

CREATE TABLE authors (
  id SERIAL PRIMARY KEY,
  name VARCHAR(100) NOT NULL,
  email VARCHAR(150) NOT NULL UNIQUE,
  bio TEXT,
  created_at TIMESTAMP NOT NULL DEFAULT NOW()
);

CREATE TABLE posts (
  id SERIAL PRIMARY KEY,
  title VARCHAR(200) NOT NULL,
  content TEXT NOT NULL,
  author_id INTEGER NOT NULL REFERENCES authors(id) ON DELETE CASCADE,
  published BOOLEAN NOT NULL DEFAULT FALSE,
  created_at TIMESTAMP NOT NULL DEFAULT NOW(),
  updated_at TIMESTAMP NOT NULL DEFAULT NOW()
);

CREATE INDEX idx_posts_author_id ON posts(author_id);
```

`posts` se elimina antes que `authors` por la foreign key que depende de ella. `IF EXISTS` evita error si la tabla ya no existe.

Seed de datos de prueba

```
-- seed.sql
INSERT INTO authors (name, email, bio) VALUES
  ('Kevin Ramirez', 'kevin@example.com', 'Desarrollador en formacion en SENA'),
  ('Ana Torres', 'ana@example.com', 'Escritora de tecnologia');

INSERT INTO posts (title, content, author_id, published) VALUES
  ('Mi primer post', 'Contenido de prueba del primer post', 1, true),
  ('Post en borrador', 'Todavia no esta listo para publicar', 1, false),
  ('Bienvenida al blog', 'Hola a todos, este es el inicio', 2, true);

psql -U tu_usuario -d miniblog -f schema.sql
psql -U tu_usuario -d miniblog -f seed.sql
```

7. `getAllAuthors` devolviendo array completo

Pregunta: ¿Cómo hacer que `getAllAuthors` devuelva un array completo?

Función básica


```
// models/authorModel.js
const pool = require('../db');

async function getAllAuthors() {
  const result = await pool.query(
    `SELECT id, name, email, bio, created_at
     FROM authors
     ORDER BY name ASC`
  );
  return result.rows;
}

module.exports = { getAllAuthors };
```

Errores comunes que causan un array incompleto

- Devolver solo la primera fila: `return result.rows[0]` en vez de `return result.rows`.
- Devolver el objeto `result` completo en vez de `.rows`.
- Un `LIMIT` olvidado en la query.
- Un `WHERE` de más que filtra sin querer.
- `client.query()` sin `client.release()` previo, dejando el pool colgado.

Caso avanzado: autores con posts anidados (JOIN + agrupación)

```
async function getAllAuthorsWithPosts() {
  const result = await pool.query(
    `SELECT
      a.id AS author_id,
      a.name,
      a.email,
      a.bio,
      p.id AS post_id,
      p.title,
      p.published
     FROM authors a
     LEFT JOIN posts p ON p.author_id = a.id
     ORDER BY a.name ASC`
  );

  const authorsMap = new Map();

  for (const row of result.rows) {
    if (!authorsMap.has(row.author_id)) {
      authorsMap.set(row.author_id, {
        id: row.author_id,
        name: row.name,
        email: row.email,
        bio: row.bio,
        posts: []
      });
    }

    if (row.post_id) {
      authorsMap.get(row.author_id).posts.push({
        id: row.post_id,
        title: row.title,
        published: row.published
      });
    }
  }

  return Array.from(authorsMap.values());
}
```

LEFT JOIN permite que autores sin posts también aparezcan. Se agrupa con un **Map** porque SQL devuelve filas planas (una por cada post).

8. Conectar rutas con sus endpoints

Pregunta: ¿Cómo conectar las rutas con su respectivo endpoint?

Definición de rutas (**routes/postRoutes.js**)

```
const express = require('express');
const router = express.Router();
const postController = require('../controllers/postController');

router.get('/', postController.getAll);
router.get('/:id', postController.getOne);
router.post('/', postController.create);
router.put('/:id', postController.update);
router.delete('/:id', postController.remove);

module.exports = router;
```

Montaje en **app.js**

```
const express = require('express');
const postRoutes = require('./routes/postRoutes');
const authorRoutes = require('./routes/authorRoutes');

const app = express();

app.use(express.json());

app.use('/api/posts', postRoutes);
app.use('/api/authors', authorRoutes);

const PORT = process.env.PORT || 3000;
app.listen(PORT, () => {
  console.log(`Servidor corriendo en http://localhost:${PORT}`);
});
```

Tabla de endpoints resultantes

Definido en postRoutes.js	Prefijo en app.js	Endpoint final
router.get('/')	/api/posts	GET /api/posts
router.get('/:id')	/api/posts	GET /api/posts/:id
router.post('/')	/api/posts	POST /api/posts
router.put('/:id')	/api/posts	PUT /api/posts/:id
router.delete('/:id')	/api/posts	DELETE /api/posts/:id

app.use(express.json()) es imprescindible para leer **req.body** en JSON.

9. Validar email con respuesta 400

Pregunta: ¿Cómo validar un email de forma eficiente, con res 400 si está mal?

Regex recomendada (simple, no RFC completa)

```
const EMAIL_REGEX = /^[^\s@]+@[^\s@]+\.[^\s@]+$/;
```

Middleware reutilizable

```
// middlewares/validateEmail.js
const EMAIL_REGEX = /^[^\s@]+@[^\s@]+\.[^\s@]+$/;

function validateEmail(req, res, next) {
  const { email } = req.body;

  if (!email) {
    return res.status(400).json({ error: 'El campo email es obligatorio' });
  }

  if (!EMAIL_REGEX.test(email)) {
    return res.status(400).json({ error: 'El formato del email no es valido' });
  }

  next();
}

module.exports = validateEmail;
```

Uso en rutas

```
// routes/authorRoutes.js
const validateEmail = require('../middlewares/validateEmail');

router.post('/', validateEmail, authorController.create);
router.put('/:id', validateEmail, authorController.update);
```

Manejo de email duplicado (constraint UNIQUE)

```
async function create(req, res) {
  try {
    const author = await authorModel.createAuthor(req.body);
    res.status(201).json(author);
  } catch (error) {
    if (error.code === '23505') {
      return res.status(409).json({ error: 'Ese email ya esta registrado' });
    }
    console.error(error);
    res.status(500).json({ error: 'Error al crear el autor' });
  }
}
```

`error.code === '23505'` es el código de error de PostgreSQL para violación de restricción UNIQUE. `409 Conflict` es el status correcto en este caso.

10. Para qué sirve `next` en un middleware

Pregunta: `function validateEmail(req, res, next)` — ¿para qué sirve `next`?

`next` es una función que Express inyecta a todo middleware. Su trabajo es decirle a Express "pasa al siguiente eslabón de la cadena".

Flujo

```
Request llega
  |
  v
validateEmail(req, res, next)
  |
  +-- Email invalido -> res.status(400)... -> CADENA TERMINA AQUI
  |
  +-- Email valido -> next() -> sigue a la siguiente funcion
                                |
                                v
                        authorController.create(req, res)
```

Por qué el `return` antes de `res.status(400)` es importante

Sin `return`, el código seguiría ejecutándose y llamaría a `next()` igual, causando el error `Cannot set headers after they are sent to the client` porque se intentarían enviar dos respuestas.

`next(error)`

Si se le pasa un argumento a `next`, Express salta directo al middleware de manejo de errores, saltándose cualquier middleware/controlador intermedio.

11. `getPostsByAuthorId`

Pregunta: ¿Cómo hacer un `getPostsByAuthorId`?

Modelo

```
async function getPostsByAuthorId(authorId) {
  const result = await pool.query(
    `SELECT id, title, content, author_id, published, created_at
     FROM posts
     WHERE author_id = $1
     ORDER BY created_at DESC`,
    [authorId]
  );
  return result.rows;
}
```

Controlador

```
async function getByAuthorId(req, res) {
  const { authorId } = req.params;

  if (isNaN(authorId)) {
    return res.status(400).json({ error: 'El authorId debe ser numerico' });
  }

  try {
    const posts = await postModel.getPostsByAuthorId(authorId);
    res.status(200).json(posts);
  } catch (error) {
  }
```

```

    console.error(error);
    res.status(500).json({ error: 'Error al obtener los posts del autor' });
  }
}

```

Un array vacío `[]` es una respuesta válida (el autor existe pero no tiene posts); no amerita 404.

Ruta y orden importante

```

router.get('/author/:authorId', postController.getByAuthorId);
router.get('/:id', postController.getOne);

```

`/author/:authorId` debe ir **antes** que `/:id`, porque Express evalúa rutas en orden y `/:id` haría match primero con `"author"` como si fuera un id.

Extra: verificar que el autor exista (404 si no)

```

const authorModel = require('../models/authorModel');

async function getByAuthorId(req, res) {
  const { authorId } = req.params;

  if (isNaN(authorId)) {
    return res.status(400).json({ error: 'El authorId debe ser numerico' });
  }

  try {
    const author = await authorModel.getAuthorById(authorId);
    if (!author) {
      return res.status(404).json({ error: 'Autor no encontrado' });
    }

    const posts = await postModel.getPostsByAuthorId(authorId);
    res.status(200).json(posts);
  } catch (error) {
    console.error(error);
    res.status(500).json({ error: 'Error al obtener los posts del autor' });
  }
}

```

12. Unir posts con autores (JOIN)

Pregunta: ¿Cómo unirlo con el id de los autores?

Query con JOIN (estructura plana)

```

async function getAllPostsWithAuthor() {
  const result = await pool.query(
    `SELECT
      p.id,
      p.title,
      p.content,
      p.published,
      p.created_at,
      a.id AS author_id,
      a.name AS author_name,
      a.email AS author_email
    FROM posts p
    JOIN authors a ON p.author_id = a.id
    ORDER BY p.created_at DESC`
  );
  return result.rows;
}

```

```
}
```

Se usan alias (**AS author_id**, **AS author_name**) porque ambas tablas tienen columnas con el mismo nombre (**id**), y sin renombrar se pisarían entre sí.

Transformar a objeto anidado

```
async function getAllPostsWithAuthor() {
  const result = await pool.query(
    `SELECT
      p.id,
      p.title,
      p.content,
      p.published,
      p.created_at,
      a.id AS author_id,
      a.name AS author_name,
      a.email AS author_email
    FROM posts p
    JOIN authors a ON p.author_id = a.id
    ORDER BY p.created_at DESC`
  );

  return result.rows.map(row => ({
    id: row.id,
    title: row.title,
    content: row.content,
    published: row.published,
    created_at: row.created_at,
    author: {
      id: row.author_id,
      name: row.author_name,
      email: row.author_email
    }
  }));
}
```

Mismo patrón en **getPostById**

```
async function getPostById(id) {
  const result = await pool.query(
    `SELECT
      p.id,
      p.title,
      p.content,
      p.published,
      p.created_at,
      a.id AS author_id,
      a.name AS author_name,
      a.email AS author_email
    FROM posts p
    JOIN authors a ON p.author_id = a.id
    WHERE p.id = $1`,
    [id]
  );
  return result.rows[0] || null;
}
```

El índice **idx_posts_author_id** ya creado en el schema es lo que hace eficiente este JOIN.

13. Parámetros del **errorHandler**

Pregunta: ¿Cómo son los parámetros para un errorHandler? ¿Por qué se hace de esa forma?

La firma

```
function errorHandler(err, req, res, next) {  
  // ...  
}
```

Express distingue el tipo de middleware por la **cantidad de parámetros** (`function.length`):

- 3 parámetros (`req`, `res`, `next`) → middleware normal.
- 4 parámetros (`err`, `req`, `res`, `next`) → middleware de manejo de errores.

`err` va siempre primero porque Express llama a la función posicionalmente.

Ejemplo completo

```
// middlewares/errorHandler.js  
function errorHandler(err, req, res, next) {  
  console.error(err.stack);  
  
  if (err.code === '23505') {  
    return res.status(409).json({ error: 'El recurso ya existe (email duplicado)' });  
  }  
  
  if (err.code === '23503') {  
    return res.status(400).json({ error: 'Referencia invalida (el author_id no existe)' });  
  }  
  
  res.status(err.status || 500).json({  
    error: err.message || 'Error interno del servidor'  
  });  
}  
  
module.exports = errorHandler;
```

`23505` = `unique_violation`, `23503` = `foreign_key_violation` (códigos nativos de PostgreSQL).

Registro en `app.js` (siempre al final)

```
app.use(express.json());  
app.use('/api/posts', postRoutes);  
app.use('/api/authors', authorRoutes);  
  
app.use(errorHandler); // SIEMPRE al final, despues de todas las rutas  
  
app.listen(3000, () => console.log('Servidor corriendo'));
```

Refactor de controladores usando `next(error)`

```
// Antes  
async function create(req, res) {  
  try {  
    const post = await postModel.createPost(req.body);  
    res.status(201).json(post);  
  } catch (error) {  
    console.error(error);  
    res.status(500).json({ error: 'Error al crear el post' });  
  }  
}  
  
// Despues (con errorHandler global)  
async function create(req, res, next) {  
  try {  
    const post = await postModel.createPost(req.body);
```

```

    res.status(201).json(post);
  } catch (error) {
    next(error);
  }
}

```

14. Crear `.env.test` y configurar Supertest

Pregunta: ¿Cómo crear correctamente un archivo `.env.test`? ¿Qué modificaciones se necesitan para usar Supertest?

`.env.test`

```

# .env.test
NODE_ENV=test
PORT=3001
DATABASE_URL=postgresql://user:password@localhost:5432/miniblog_test

```

Base de datos separada (`miniblog_test`) para no ensuciar los datos de desarrollo:

```

psql -U user -c "CREATE DATABASE miniblog_test;"
psql -U user -d miniblog_test -f schema.sql

```

Cargar el `.env` correcto según el entorno

```

// db.js
require('dotenv').config({
  path: process.env.NODE_ENV === 'test' ? '.env.test' : '.env'
});

```

Separar `app.js` de `server.js`

```

// app.js – SOLO configura la app, NO la levanta
const express = require('express');
const postRoutes = require('./routes/postRoutes');
const authorRoutes = require('./routes/authorRoutes');
const errorHandler = require('./middlewares/errorHandler');

const app = express();

app.use(express.json());
app.use('/api/posts', postRoutes);
app.use('/api/authors', authorRoutes);
app.use(errorHandler);

module.exports = app;

// server.js – este es el que SI levanta el servidor
const app = require('./app');

const PORT = process.env.PORT || 3000;
app.listen(PORT, () => {
  console.log(`Servidor corriendo en http://localhost:${PORT}`);
});

```

Supertest necesita el objeto `app` sin `.listen()` real, para simular peticiones HTTP en memoria.

Instalación

```
npm install --save-dev supertest jest
```


package.json

```
{
  "scripts": {
    "start": "node server.js",
    "test": "NODE_ENV=test jest --runInBand"
  }
}
```

--runInBand corre los tests secuencialmente, evitando que se pisen datos en la misma base de datos compartida.

Primer test

```
// tests/posts.test.js
const request = require('supertest');
const app = require('../app');

describe('GET /api/posts', () => {
  it('deberia devolver un array de posts con status 200', async () => {
    const response = await request(app).get('/api/posts');

    expect(response.status).toBe(200);
    expect(Array.isArray(response.body)).toBe(true);
  });
});

describe('POST /api/posts', () => {
  it('deberia devolver 400 si faltan campos obligatorios', async () => {
    const response = await request(app)
      .post('/api/posts')
      .send({ title: 'Solo titulo, sin content ni authorId' });

    expect(response.status).toBe(400);
    expect(response.body).toHaveProperty('error');
  });
});
```

Estructura final del proyecto

```
proyecto/
  ■■■ .env
  ■■■ .env.test
  ■■■ app.js
  ■■■ server.js
  ■■■ db.js
  ■■■ controllers/
  ■■■ models/
  ■■■ routes/
  ■■■ middlewares/
  ■■■ tests/
    ■■■ posts.test.js

npm test
```

15. Error de deploy en Railway

Pregunta: Error al hacer deploy en Railway desde GitHub:

```
TypeError: Cannot read properties of undefined (reading 'includes')
    at Object.<anonymous> (/app/src/config/db.js:4:29)
    ...
```

Causa

El código lee una variable de entorno (probablemente `DATABASE_URL`) que no está definida en Railway, y llama `.includes()` sobre ella:

```
const pool = new Pool({
  connectionString: process.env.DATABASE_URL,
  ssl: process.env.DATABASE_URL.includes('railway') ? { rejectUnauthorized: false } : false,
});
```

Railway no usa el archivo `.env` local; las variables deben configurarse directamente en la plataforma.

Solución

- Entrar al proyecto en Railway.
- Si se usa el plugin de PostgreSQL de Railway, ir al servicio de la API → pestaña Variables → referenciar la variable del servicio de base de datos (`Postgres.DATABASE_URL`) o copiar el valor manualmente.
- Si es un proveedor externo (Supabase, Neon, etc.), agregar `DATABASE_URL` manualmente con la connection string real de producción.
- Redeploy (automático o manual).

Código más defensivo

```
// src/config/db.js
const { Pool } = require('pg');

if (!process.env.DATABASE_URL) {
  throw new Error('DATABASE_URL no esta definida. Verifica tus variables de entorno.');
```

```
}

const pool = new Pool({
  connectionString: process.env.DATABASE_URL,
  ssl: process.env.DATABASE_URL.includes('railway')
    ? { rejectUnauthorized: false }
    : false,
});

module.exports = pool;
```

Este chequeo da un mensaje de error claro en vez del stack trace críptico original.

Resultado: Kevin confirmó que el problema era efectivamente la variable de entorno faltante en Railway, y quedó solucionado.